Lecture 3: The MapReduce Paradigm

DSL351 / DSP352: Big Data Analytics

Amit Kumar Dhar

IIT Bhilai

January 8, 2026

Agenda

1. Introduction to MapReduce
2. Functional Programming Roots
3. Data Locality
4. The Life of a MapReduce Job
5. Real Life Analogy
6. Case Study: Word Count
7. Fault Tolerance
8. Summary

Introduction to MapReduce

Origins

- **Problem (Pre-2004):** Google needed to index the entire internet.
- **Challenge:** Processing petabytes of data on commodity hardware.
- **Issues:** Handling failures, parallelization, network bandwidth.
- **Solution:** *"MapReduce: Simplified Data Processing on Large Clusters"* (Dean & Ghemawat, 2004).

A programming model for processing large data sets with a parallel, distributed algorithm on a cluster.

The Core Concept

MapReduce divides tasks into two phases:

1. **Map Phase:** Processes input records independently. Parallelizable.
2. **Reduce Phase:** Aggregates results from the Map phase.

"Hides the messy details of parallelization, fault-tolerance, data distribution and load balancing."

Functional Programming Roots

Roots in Lisp/Functional Programming

MapReduce is inspired by the `map` and `reduce` primitives in functional languages.

- **Pure Functions:** No side effects. Output depends only on input.
- **Immutable Data:** Input data is never modified.
- **High-Order Functions:** Functions that take functions as arguments.

The `map` Function

Definition: Applies a function f to every element in a list.

$$map(f, [x_1, x_2, ..., x_n]) \rightarrow [f(x_1), f(x_2), ..., f(x_n)]$$

Example (Python):

```
nums = [1, 2, 3, 4]
squared = map(lambda x: x*x, nums)
# Result: [1, 4, 9, 16]

```

Key: Each operation is independent. Perfect for parallelism.

The `reduce` Function

Definition: Combines elements of a list using a binary operator $\oplus$.

$$reduce(\oplus, [x_1, x_2, ..., x_n]) \rightarrow x_1 \oplus x_2 \oplus ... \oplus x_n$$

Example (Python):

```python
from functools import reduce
nums = [1, 2, 3, 4]
sum = reduce(lambda x, y: x+y, nums)
# Result: 10 ((1+2)+3)+4

```

Key: Aggregates results.

Data Locality

The Bottleneck: Network I/O

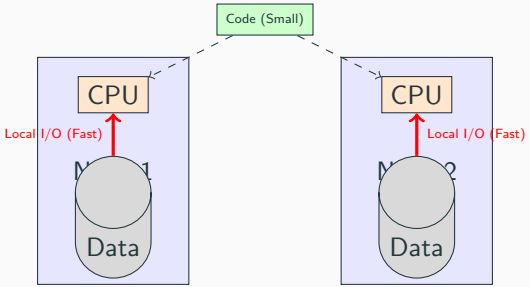
In traditional HPC (High Performance Computing):

- Compute Nodes $\neq$ Storage Nodes (SAN/NAS).
- Data is moved to the compute node.

Big Data Reality:

- Disk I/O speed: $\sim$100 MB/s.
- Network speed: $\sim$1 Gbps ($\sim$125 MB/s) [Shared].
- Moving Petabytes is slow!

Principle: Move Computation to Data



Instead of moving 1TB of data, move 10KB of code (jar file) to the node where data resides.

The Life of a MapReduce Job

Workflow Overview

A MapReduce job goes through distinct phases:

1. **Input Splits:** Logical representation of data.
2. **Mapping:** The user-defined logic.
3. **Shuffling & Sorting:** The framework magic.
4. **Reducing:** The aggregation logic.
5. **Output:** Writing results.

Phase 1: Input Splits

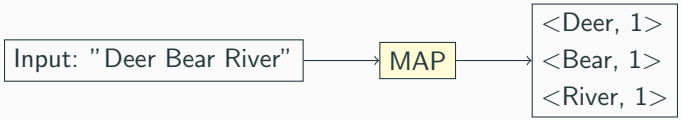
- HDFS blocks are typically 128MB.
- InputSplit describes a unit of work (usually 1 Split = 1 Block).
- **Example:** A 500MB file has 4 splits (128, 128, 128, 116).
- **RecordReader:** Reads data from Split and converts to `<Key, Value>` pairs.

Phase 2: The Map Phase

- **Input:** <K1, V1> (e.g., <Offset, LineOfText>)
- **Function:** Process each record. Filter, Parse, Transform.
- **Output:** List of <K2, V2> (Intermediate Keys).

Map tasks run in parallel on different nodes (Data Locality).

Illustration: Map Phase



Phase 3: The Combiner (Optional)

- Known as "Mini-Reducer".
- Runs locally on the Mapper node.
- **Goal:** Pre-aggregate data to reduce network traffic.
- **Example:** instead of sending `<The, 1>` 100 times, send `<The, 100>` once.

Phase 4: Partitioning

- Determines which Reducer gets which key.
- **Default:** `Hash(Key) mod NumReducers`.
- Ensures all occurrences of the same Key go to the *same* Reducer.
- Crucial for correctness.

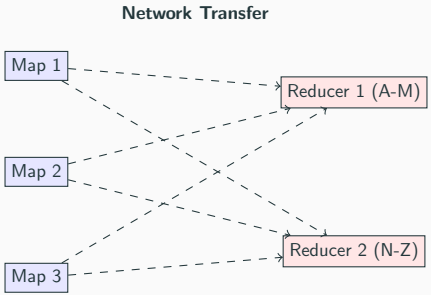
Phase 5: Shuffle and Sort

The Heart of MapReduce

- **Shuffle:** Physical transfer of data from Mappers to Reducers over the network.
- **Sort:** Reducers sort incoming data by Key.
- **Grouping:** Values for the same key are grouped: `<Key, [List of Values]>`.

This is the most expensive phase (Network + Disk I/O).

Illustration: Shuffle



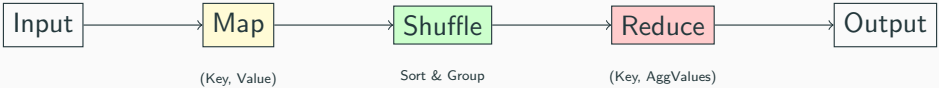
Phase 6: The Reduce Phase

- **Input:** `<K2, [V2, V2, ...]>`
- **Function:** Aggregate the list of values. Sum, Max, Concat.
- **Output:** `<K3, V3>` (Final Result).

Phase 7: Output

- Results are written back to HDFS.
- Typically using `TextOutputFormat` (Tab separated).
- First copy written locally, then replicated to other nodes (Pipeline).

Full Workflow



Real Life Analogy

Counting Votes

Scenario: National Election.

- **Split:** Each polling station is an input split.
- **Map:** Local officials count votes in their box (e.g., "A: 10, B: 5").
- **Combiner:** Sum up local box totals.
- **Shuffle:** Send all results for Candidate A to Center A, Candidate B to Center B.
- **Reduce:** Center A sums up all incoming district counts.
- **Output:** Final National Result.

Case Study: Word Count

The "Hello World" of MapReduce

Goal: Count frequency of every word in a large text corpus.

Input:

- File 1: "Deer Bear River"
- File 2: "Car Car River"
- File 3: "Deer Car Bear"

Word Count: Map

Mapper Logic:

- For each word in line, emit `<Word, 1>`.

Output:

Node 1

¡Deer, 1¿
¡Bear, 1¿
¡River, 1¿

Node 2

Node 3

Word Count: Shuffle & Sort

System groups values by Key.

Intermediate (at Reducers):

- **Bear:** [1, 1]
- **Car:** [1, 1, 1]
- **Deer:** [1, 1]
- **River:** [1, 1]

Word Count: Reduce

Reducer Logic:

- Sum the list of values.

Final Output:

- Bear: 2
- Car: 3
- Deer: 2
- River: 2

Fault Tolerance

Failures are Norm, Not Exception

In a cluster of 1,000 nodes, expect 1-2 failures per day.

How MapReduce Handles It:

- **No Restarting Job:** We do not restart the whole 10-hour job if one node dies.
- **Task Level Recovery:** Only the *failed task* is re-executed.

Worker Failure

- Master (YARN ResourceManager) sends heartbeats.
- If no heartbeat for X seconds, node is dead.
- **Map Task Failure:** Reschedule on another node containing a replica of input data.
- **Reduce Task Failure:** Reschedule on another node. Re-fetch data from Mappers.

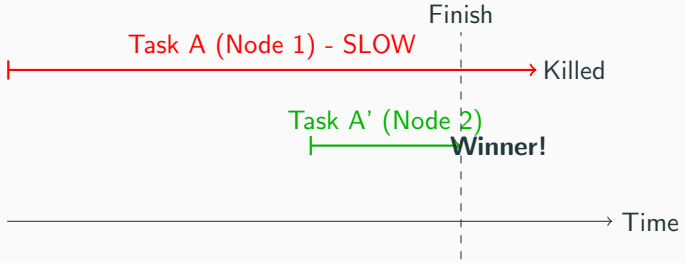
Speculative Execution (Stragglers)

Problem: One slow machine (bad disk, high load) delays the entire job.

Solution:

1. Master notices a task is slower than average.
2. Launches a **backup (speculative)** copy of the same task on another node.
3. The one that finishes first "wins". The other is killed.

Illustration: Speculative Execution



Summary

Key Takeaways

- **Abstraction:** MapReduce simplifies parallel programming.
- **Data Locality:** Move code to data to save bandwidth.
- **Scalability:** Linearly scalable (Add more nodes = Faster processing).
- **Fault Tolerance:** Handles failures automatically via retries and speculative execution.

Questions?

Next Class: Writing MapReduce Code